INTRODUCTION TO SOFTWARE ENGINEERING

Bahria Transport Management system

Software Requirements Specification

Version 1.0
May 17,2025

Minahil Khushid Chaudhry
Zunaira
Khaula Khalid

Software Engineers

Prepared for
Sen 120- Introduction to Software Engineering
Instructor: Rafia Mumtaz
Spring 2025

Revision History

Date	Description	Author	Comments
17-05-25	Version 1.0 Created	Minahil Chaudhry	Initial Draft Submission
		Zunaira	
		Khaula Khalid	

Document Approval

The following Software Requirements Specification has been accepted and approved by the following:

Signature	Printed Name	Title	Date
	Engineer Rafia	Instructor, SEN-120	

Table of Content

Revision History..... ii
Document Approval..... ii
1. Introduction..... 1
1.1 Purpose.....1
1.2 Scope.....1
1.3 Definitions, Acronyms, and Abbreviations.....1
1.4 References.....1
1.5 Overview.....1
2. General Description.....2
2.1 Product Perspective.....2
2.2 Product Functions.....2
2.3 User Characteristics.....2
2.4 General Constraints.....2
2.5 Assumptions and Dependencies.....2
3. Specific Requirements.....2
3.1 External Interface Requirements.....3
3.1.1 User Interfaces.....3
3.1.2 Hardware Interfaces.....3
3.1.3 Software Interfaces.....3
3.1.4 Communications Interfaces.....3
3.2 Functional Requirements.....3
3.3 Use Cases.....3
3.4 Classes / Objects.....3
3.5 Non-Functional Requirements.....4
3.5.1 Performance.....4
3.5.2 Reliability.....4
3.5.3 Availability.....4
3.5.4 Security.....4
3.5.5 Maintainability..... 4
3.5.6 Portability..... 4
3.6 Inverse Requirements..... 4
3.7 Design Constraints..... 4
3.8 Logical Database Requirements..... 4
3.9 Other Requirements..... 4

Table of Content

4. Analysis Models.....4
4.1 Sequence Diagrams.....5
4.3 Data Flow Diagrams (DFD).....5
4.2 State-Transition Diagrams (STD).....5
5. Change Management Process.....5
A. Appendices.....5
A.1 Appendix 1.....5
A.2 Appendix 2.....5

1. Introduction

The purpose of this document is to define the requirements for the Bahria Transportation Management System, which will automate the allocation and scheduling of buses for students at Bahria University. This document serves as a reference for software engineers, testers, and project stakeholders.

1.1 Purpose

1. Define the functional and non-functional requirements for the Bahria Bus Rental Management System.
2. Provide a foundation for system design, implementation, and testing activities.
3 .Document key constraints, assumptions, and evolution requirements for future development.

1.2 Scope

The software product to be developed is the Bahria Bus Rental Management System, which will:

- Automate student registration for bus services
- Manage bus and route information
- Assign students to buses based on their location
- Provide route and schedule information to students
- Allow administrators to manage the system

The system will not include:

- Payment processing (as this is a university service)
- Real-time GPS tracking (in initial version)
- Mobile app interface (future enhancement)

The software product to be produced is the Bahria Bus Rental Management System, a console-based application that automates student transportation management at Bahria University.

The system will be deployed by Bahria University's administration to:

1. Replace manual paper-based bus allocation processes
2. Centralize student transportation records
3. Provide real-time bus schedule information to students

(a) Benefits, Objectives, and Goals

Category	Precise Specification
Administrative Efficiency	Reduce bus assignment processing time from 3-5 manual working days to <5 minutes per student
Student Convenience	Provide route information access within 10 seconds of login via CLI
Data Accuracy	Eliminate 100% of manual entry errors in bus assignments through automated validation
Resource Optimization	Achieve 95%+ bus seat utilization through algorithmic assignment based on student addresses
Transparency	Display assigned bus details to students immediately after registration

b) System Boundaries

Included Capabilities:

- Student registration/login (CLI interface)
- Bus/route management (admin-only functions)
- Automated assignment based on address/route mapping
- Real-time seat availability checks

Excluded Capabilities (Future Enhancements):

- Mobile app interface
- Persistent database storage
- GPS bus tracking

1.3 Definitions, Acronyms & Abbreviations

SRS	*Software Requirements Specification*
CLI	*Command Line Interface*
DFD	*Data Flow Diagram*
UML	*Data Flow Diagram*

1.4 References

1. ISE Assignment #1: Requirements Identification
2. ISE Assignment #2: System Analysis (Use Case and DFDs)
3. ISE Assignment #3: System Design (Class Diagrams, Activity diagram, Sequence Diagrams)
4. IEEE Software Requirements Specification guide
5. Tools Used: Visual Paradigm, Canva, Word, Trello, Mermaid.AI

1.5 Overview

This document is organized into five main sections:

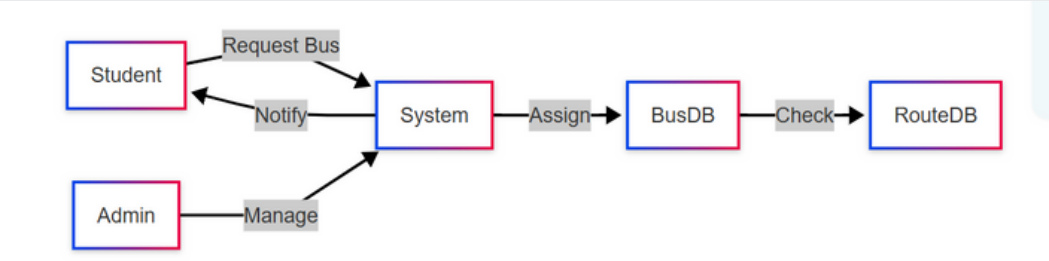
- Introduction
- General description of the system
- Specific requirements (functional and non-functional)
- Analysis models
- Change management process

2. General Description

This section provides a high-level view of the system, its key components and external relationships . it does not describe individual requirements but sets the foundation for them.

2.1 Product Perspective

The system is a standalone console-based application developed in C++. It replaces the manual process of bus assignment and scheduling at Bahria University.



2.2 Product Functions

Key functions include:

- Student registration and login
- Bus and route management
- Student-bus assignment
- Route information display
- Basic reporting

2.3 User Characteristics

Primary users:

- **Students:** Need to register, view assignments, and check schedules
- **Admin**: Manages all system data and assignments

2.4 General Constraints

- Console-based interface only
- Limited to GCC or Visual Studio compilers
- No authentication security beyond name/ID matching

2.5 Assumptions and Dependencies

- Students will provide accurate information
- Bus capacity and routes are predefined
- System runs on Windows OS
- Requires C++ compiler

3. Specific Requirements

This section defines the implementable requirements (D-requirements) that will guide the design, coding, and testing of the Bahria Transport Management System.

Requirements are structured hierarchically (e.g., 3.4.2.1) under:

1. External Interfaces (CLI, data storage)
2. Functional Requirements (Student registration, bus assignment)
3. Use Cases (Login, admin workflows)
4. Classes/Objects (Student, Bus, Route)
5. Non-Functional Requirements (Performance, reliability)

3.1 External Interface Requirements

3.1.1 User Interfaces

- Console-based menu system
- Formatted text output using setw
- Input via cin and getline()

3.1.2 Hardware Interfaces

- Standard PC hardware
- No special requirements

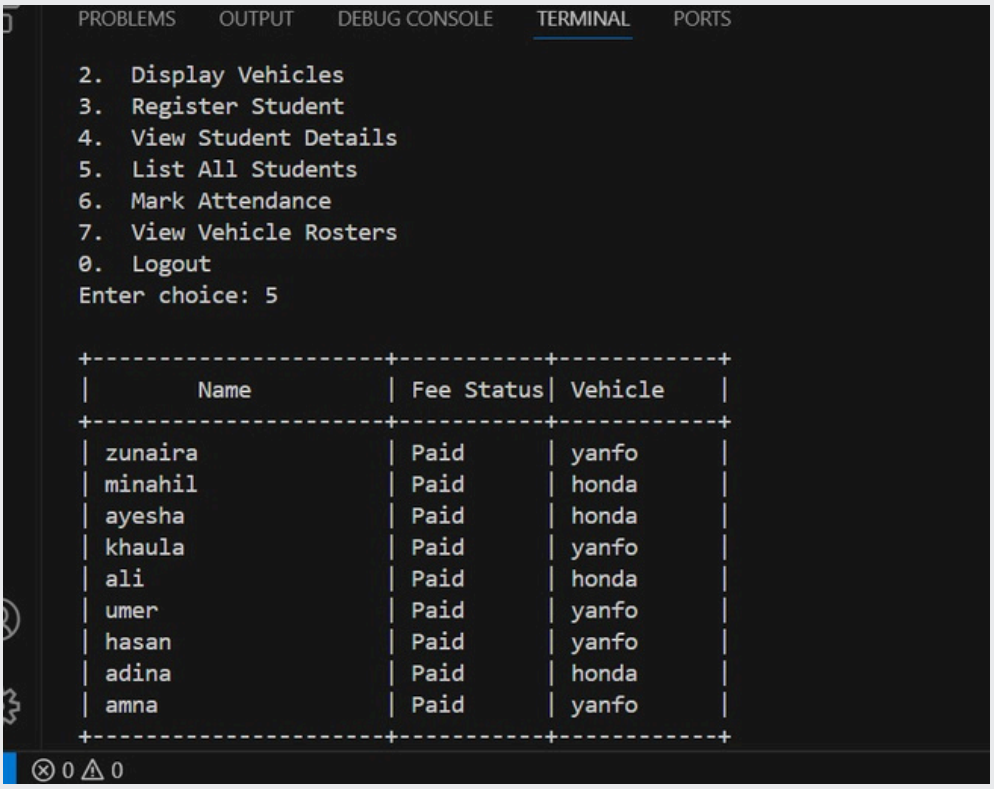
3.1.3 Software Interfaces

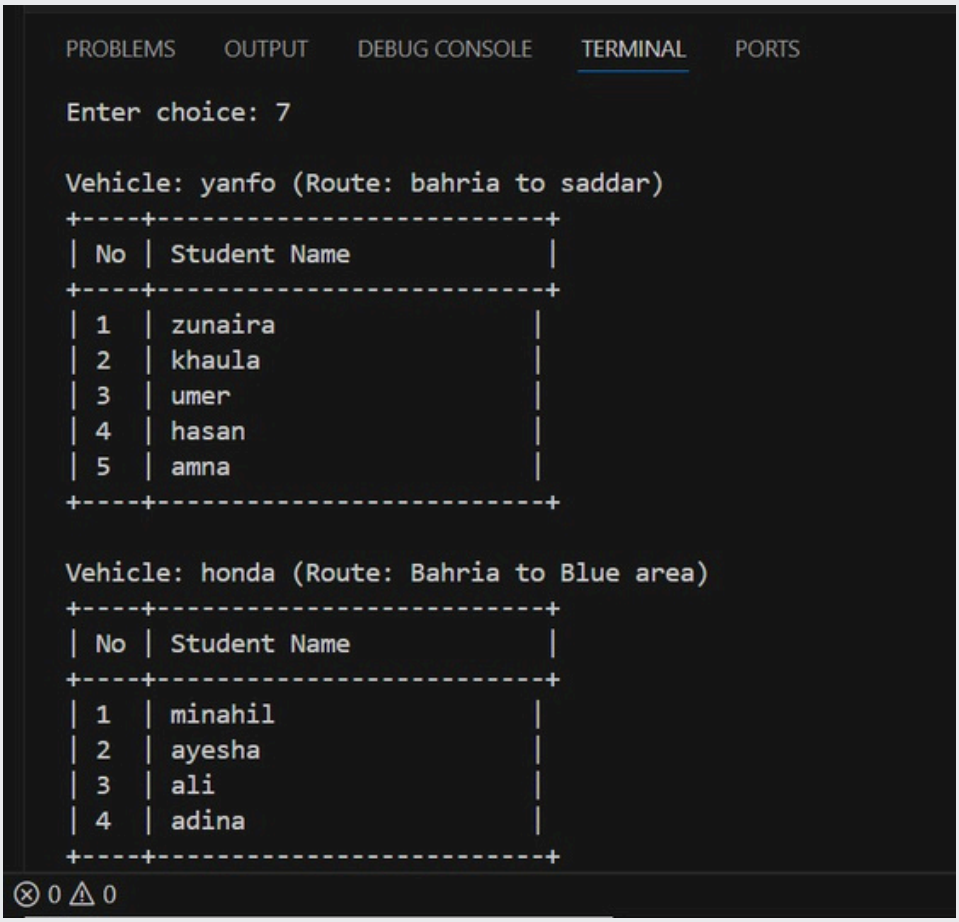
- C++ standard library
- No external database (in-memory storage only)

3.1.4 Communications Interfaces

- None in initial version

User Interfaces





3.2 Functional Requirements

3.2.1 Student Registration
3.2.2 Student Login
3.2.3 Bus Assignment
3.2.4 Route Display
3.2.5 Admin Report Generation

3.2.1 <Feature 1> Student Registration

3.2.1.1 Introduction: Allows new students to register for bus services.

3.2.1.2 Inputs:

- Student name
- Student ID
- Department
- Contact number
- Address

3.2.1.3 Processing:

- Validate inputs (reject duplicates/invalid formats)
- Store student data in memory

3.2.1.4 Outputs:

- Confirmation message
- Assigned bus details (if available)

3.2.1.5 Error Handling:

- Show "Invalid input" for incorrect data
- Display "No buses available" if assignment fails

3.2.2 <Feature 2> Student Login

3.2.2.1 Introduction: Authenticates students to access their bus information.

3.2.2.2 Inputs:

- Student name
- Student ID

3.2.2.3 Processing:

- Verify credentials against stored records

3.2.2.4 Outputs:

- Student dashboard with bus/route details

3.2.2.5 Error Handling:

- Show "Login failed" after 3 attempts

3.2.3 <Feature 3>Bus Assignment

3.2.3.1 Introduction

Automatically assigns students to buses based on address and availability.

3.2.3.2 Inputs

- Student address
- Current bus capacity

3.2.3.3 Processing

- Match student address to predefined routes
- Assign to bus with available seats

3.2.3.4 Outputs

- Bus assignment confirmation
- Route details

3.2.3.5 Error Handling

- Flag for admin if no buses available

3.2.4 <Feature 4> Route Display

3.2.4.1 Introduction: Shows available bus routes and schedules.

3.2.4.2 Inputs:

- None (system-triggered)

3.2.4.3 Processing:

- Retrieve route data
- Format with setw

3.2.4.4 Outputs:

- Formatted route table

3.2.4.5 Error Handling:

- Display "No routes defined" if empty

3.2.5 <Feature 5> Admin Report Generation

3.2.5.1 Introduction: Generates bus utilization reports.

3.2.5.2 Inputs:

- Date range (optional)

3.2.5.3 Processing:

- Calculate seat occupancy
- Format as PDF/text

3.2.5.4 Outputs:

- Report with utilization percentages

3.2.5.5 Error Handling:

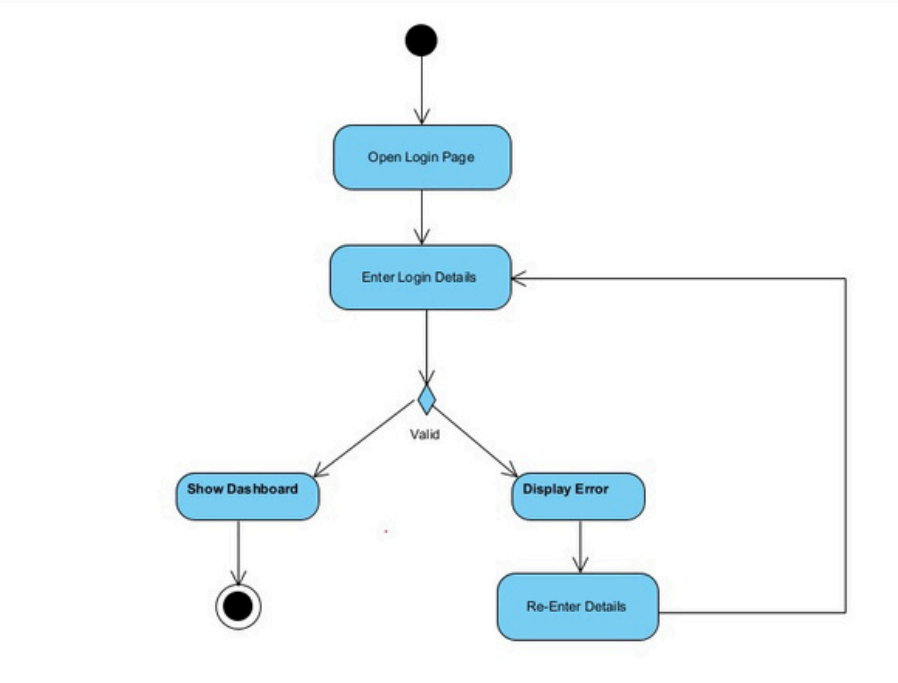
- Show "No data" for invalid dates

3.3 Use Cases

3.3.1 Student Login
3.3.2 Bus Registration
3.3.3 View Bus Routes
3.3.4 Admin-Managed Bus Assignment
3.3.5 Seat Availability Check

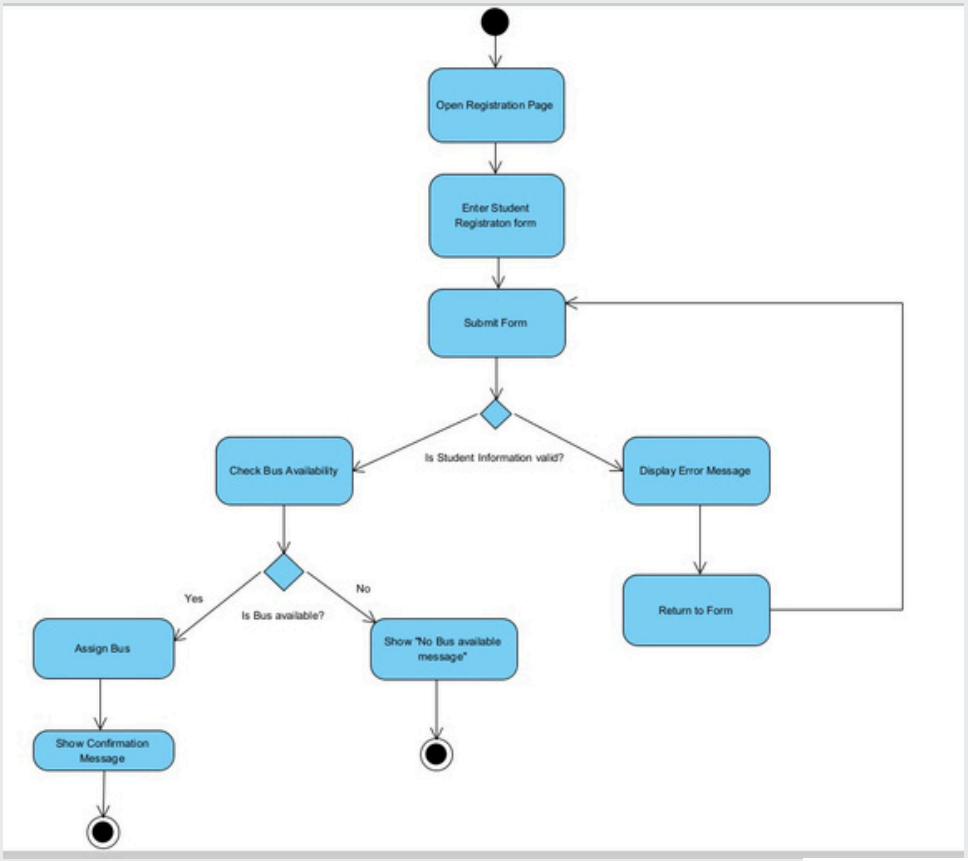
3.3.1 Use Case #1: Student Login

1. Student opens login page
2. Enters name and ID
3. System validates credentials
4. If valid, shows dashboard
5. If invalid, shows error and allows retry



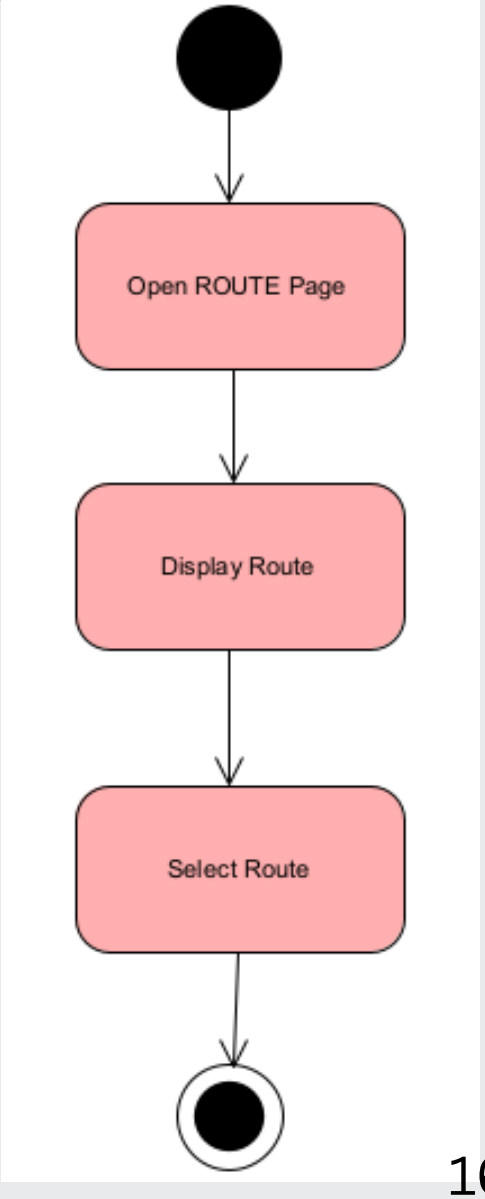
3.3.2 Use Case #2: Bus Registration

1. Student submits registration form
2. System validates information
3. Checks bus availability
4. Assigns bus if available
5. Shows confirmation



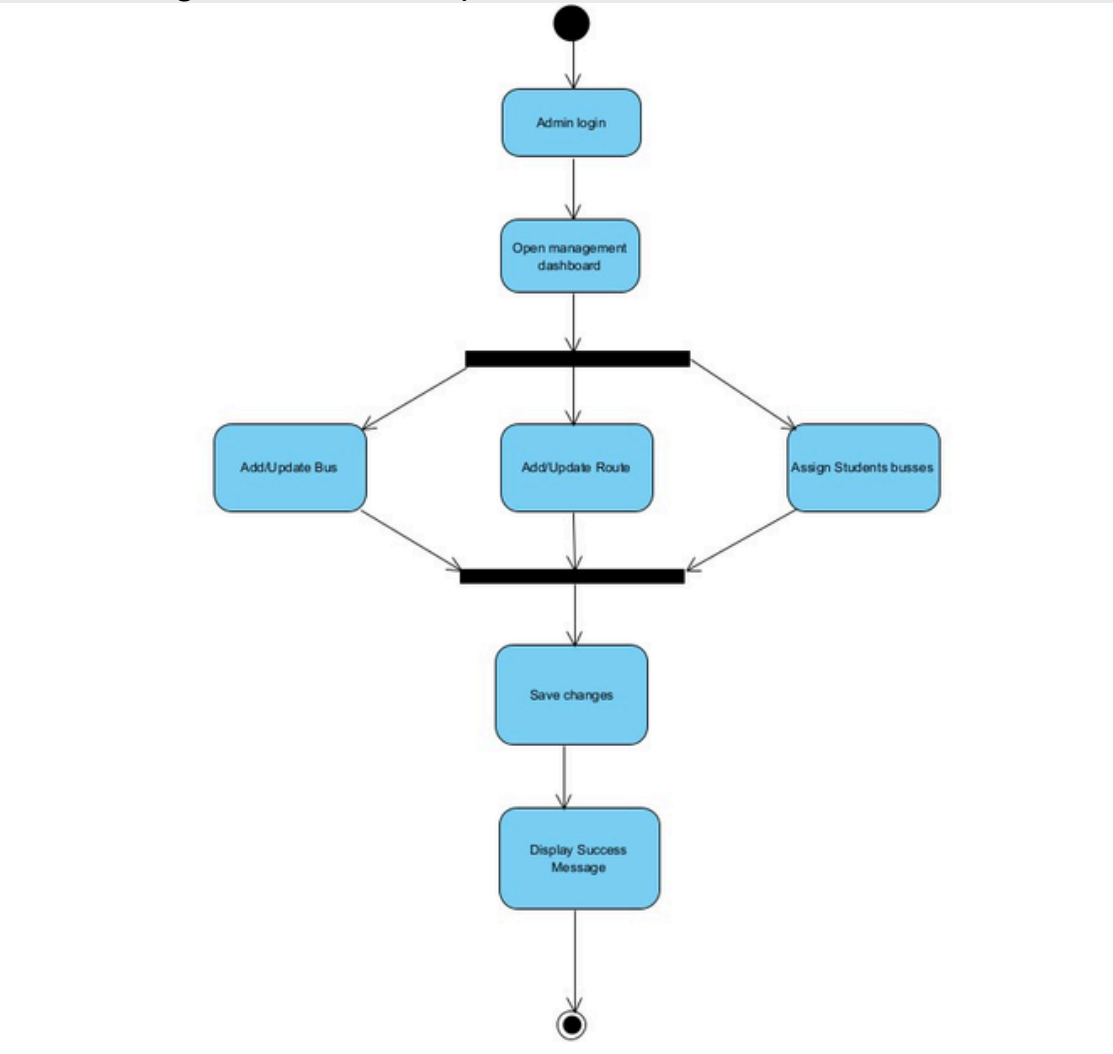
3.3.3 **Use Case #3:** View Bus Routes

1. Student requests route list
2. System retrieves predefined routes/stops
3. Displays formatted schedule (using setw)



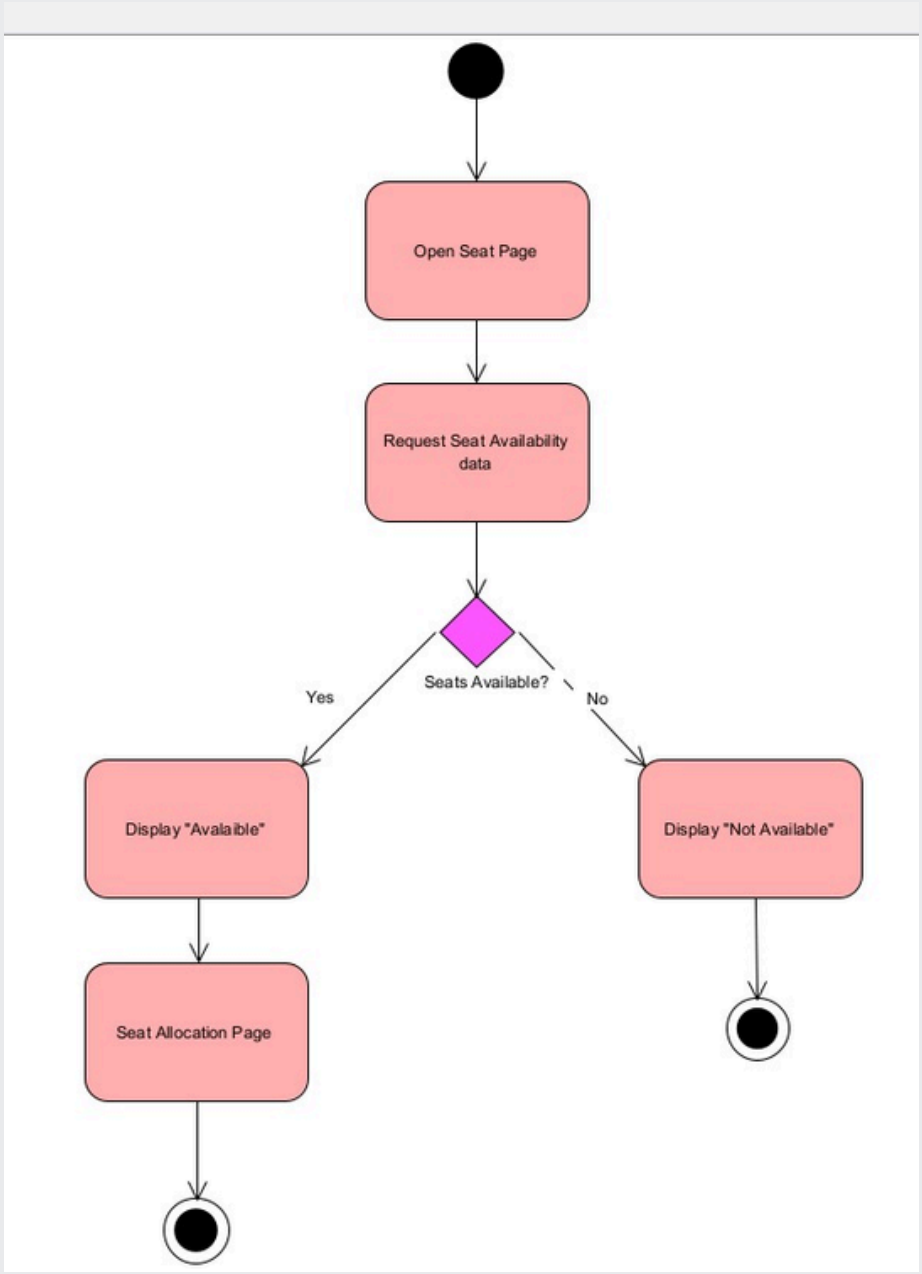
3.3.4 Use Case #4: Admin-Managed Bus Assignment

1. Admin logs in with elevated privileges
2. Selects student and bus manually
3. System overrides auto-assignment
4. Saves changes and confirms update



3.3.5 **Use Case #5:** Seat Availability Check

1. Student selects "Check Seats" option
2. System queries Bus.capacity attribute
3. Returns "Available/Full" status




Bahria Transportation Management System
M
Login
User Must Login to perform a task
Register for bus service
View bus Assigned
Add/Update Info
Send Notifications
a
Login
Manage Student Data
Assign Bus
Manage Bus routes
Add/Update bus schedules
Student (Primary Actor)
Admin

3.4 Classes / Objects

3.4.1 User Class (Base Class)
3.4.2 Student Class (Extends User)
3.4.3 Admin Class (Extends User)
3.4.4 Bus Class (Core Functionality)
3.4.5 Route Class
3.4.6 Notification Class

3.4.1 User Class (Base Class)

3.4.1.1 Attributes

· userID (string): Unique identifier
· name (string): Full name
· email (string): Contact email
· password (string)

3.4.1.2 Functions

· login(): Validates credentials (name + ID)
· logout(): Ends session

3.4.2 Student Class (Extends User)

3.4.2.1 Attributes

· department (string): Academic department
· address (string): Residential address
· assignedBus (string): Linked bus ID (Assignment #3, p2)

3.4.2.2 Functions

· register(): Saves student details → Returns confirmation
· viewBusAssignment(): Displays assigned bus/route
· updateDetails(): Modifies address/contact → Validates inputs

3.4.3 Admin Class (Extends User)

3.4.3.1 Attributes

· accessLevel (string): "Admin" (default)

3.4.3.2 Functions

· addBus(): Creates new bus record
· assignBusToStudent(): Manual override
· generateReport(): Produces usage analytics

3.4.4 Bus Class (Core Functionality)

3.4.4.1 Attributes

· busID (string): Unique identifier
· capacity (int): Max seats

· driver (string): Driver name
· status (string): "Active/Maintenance/Full"

3.4.4.2 Functions

· hasSeats(): Returns boolean if seats available
· assignSeats(): Updates capacity count

3.4.5 Route Class

3.4.5.1 Attributes

· routeID (string): Unique identifier
· stops (list): Array of stop names
· schedule (string): Timings

3.4.5.2 Functions

· updateSchedule(): Modifies timings

3.4.6 Notification Class

3.4.6.2 Functions

sendConfirmation()
displayAlert()
(e.g., delays/cancellations).

3.5 Non-Functional Requirements

3.5.1 Performance

· Response time < 2 seconds for all operations
· Support up to 1000 student records

3.5.2 Reliability

· System should operate without crashes
· Data validation for all inputs

3.5.3 Availability

· Available during university hours
· Minimal downtime

3.5.4 Security

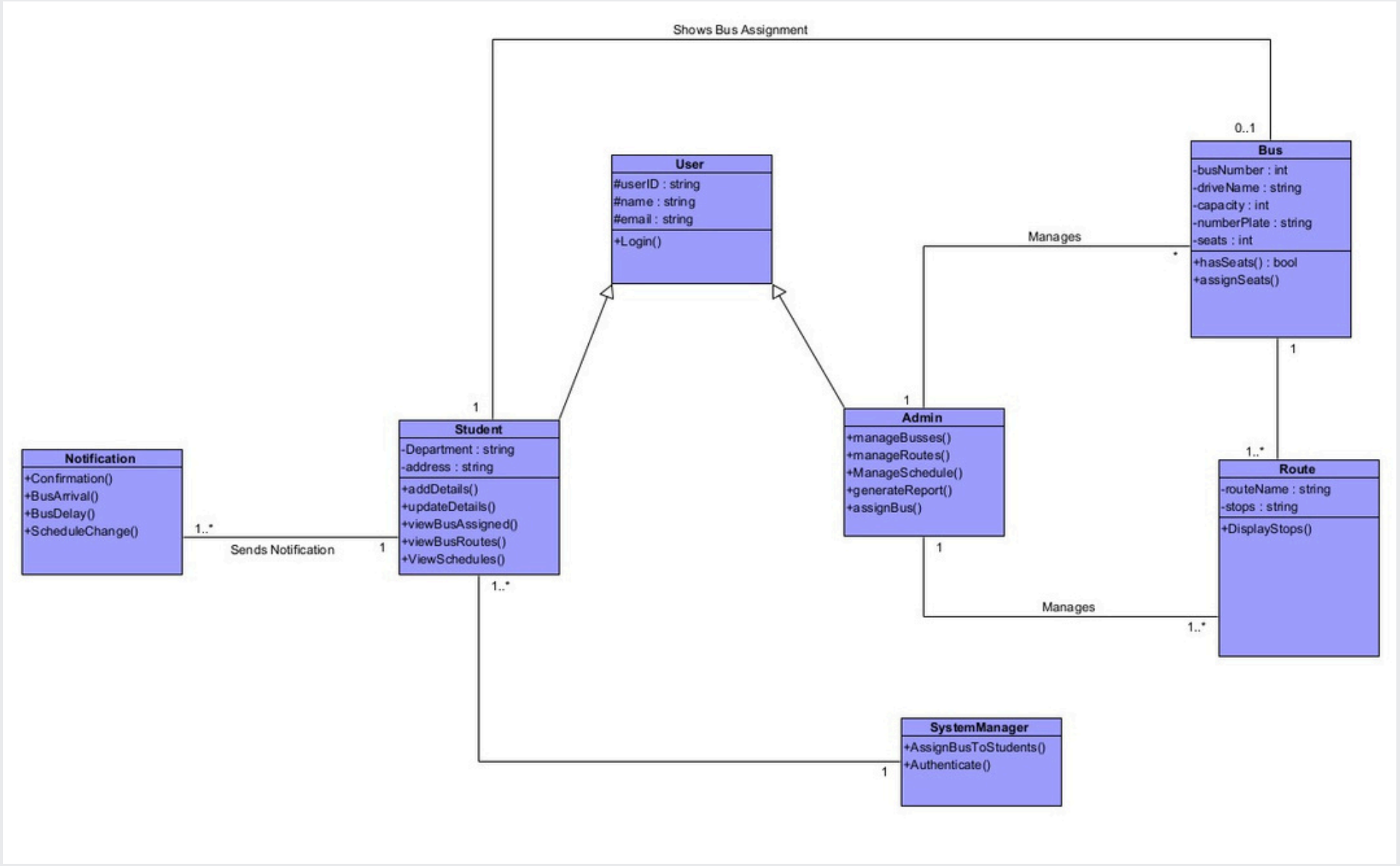
· Basic name/ID authentication
· No password protection

3.5.5 Maintainability

· Modular code structure
· Documented source code

3.5.6 Portability

· Works on Windows with GCC/Visual Studio


Shows Bus Assignment
0..1
Bus
-busNumber : int
-driveName : string
-capacity : int
-numberPlate : string
-seats : int
+hasSeats() : bool
+assignSeats()
User
#userID : string
#name : string
#email : string
+Login()
Manages
*
1
1
Admin
+manageBusses()
+manageRoutes()
+ManageSchedule()
+generateReport()
+assignBus()
1..*
Route
-routeName : string
-stops : string
+DisplayStops()
1
Manages
1..*
1
Student
-Department : string
-address : string
+addDetails()
+updateDetails()
+viewBusAssigned()
+viewBusRoutes()
+ViewSchedules()
1..*
Notification
+Confirmation()
+BusArrival()
+BusDelay()
+ScheduleChange()
1..*
Sends Notification
1
SystemManager
+AssignBusToStudents()
+Authenticate()
1

3.6 Inverse Requirements

· System should NOT store data persistently
· No web or mobile interface required

3.7 Design Constraints

2. Must use C++
3. Console interface only
4. No external databases

3.8 Logical Database Requirements

· In-memory data structures
· Arrays/lists for student and bus data
· No formal database in initial version

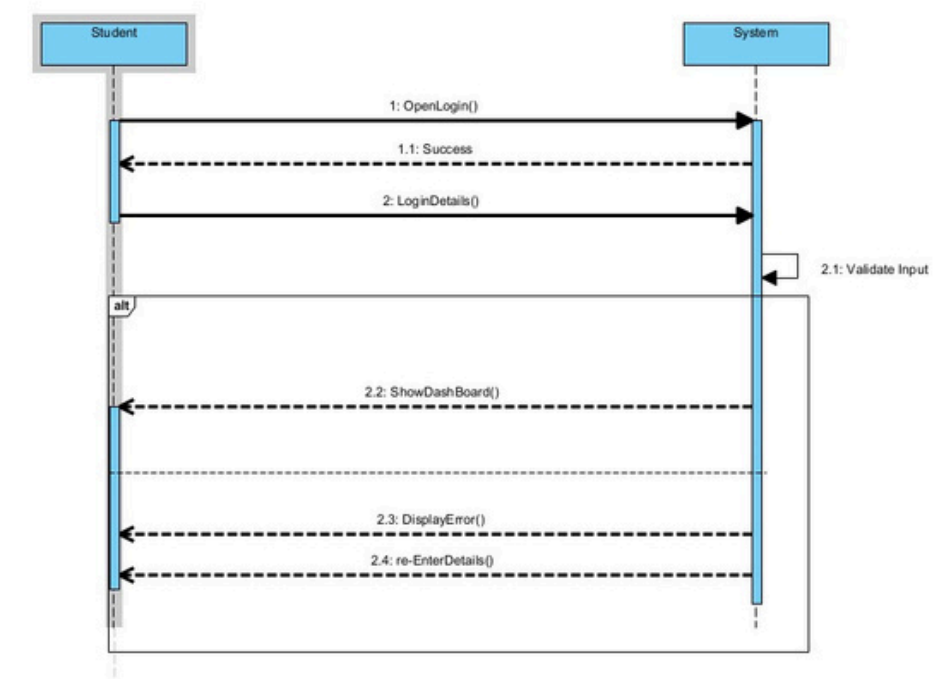
3.9 Other Requirements

· Clear screen after operations (system("cls"))
· Formatted output using setw

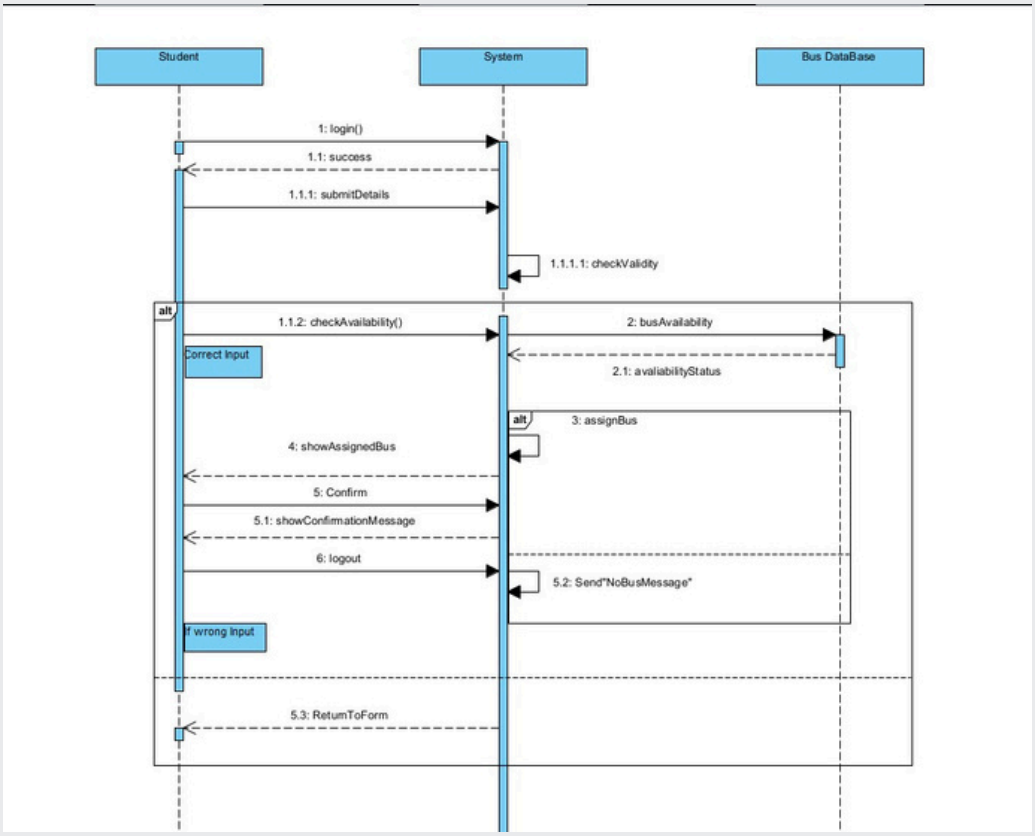
4. Analysis Models

4.1 Sequence Diagrams

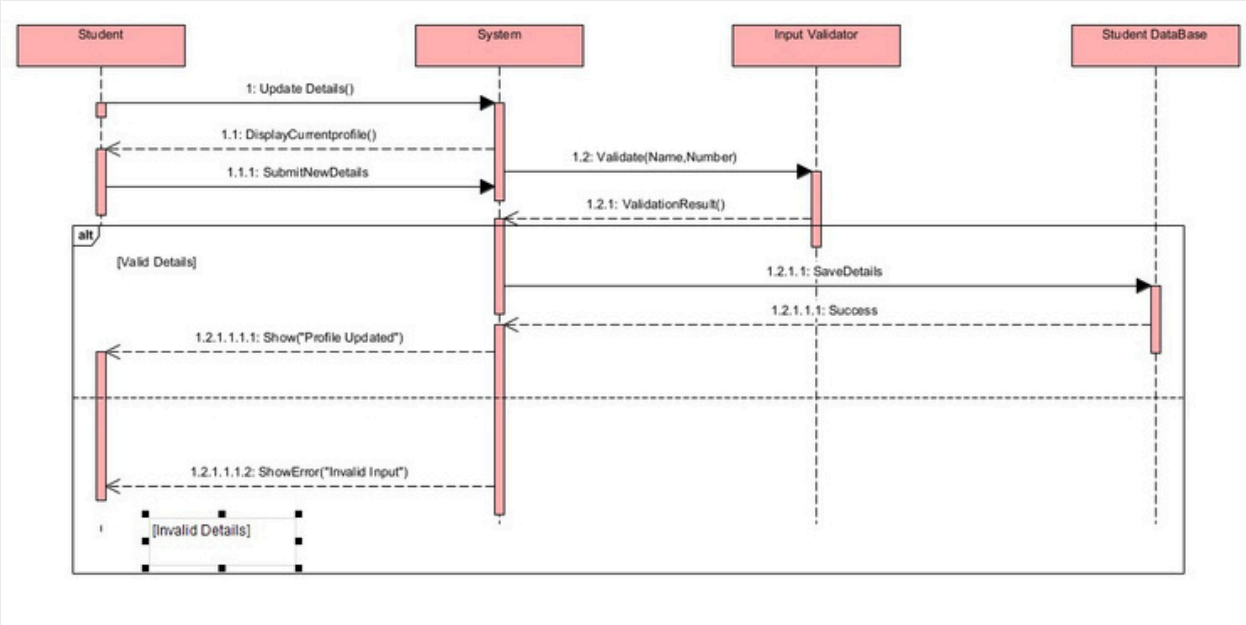
4.1.1 Student Login



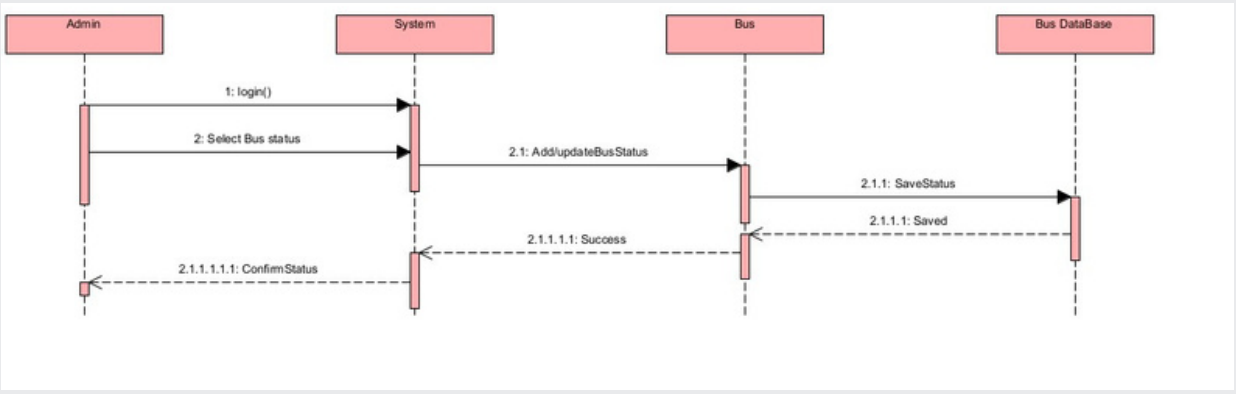
4.1.2 Bus Registration



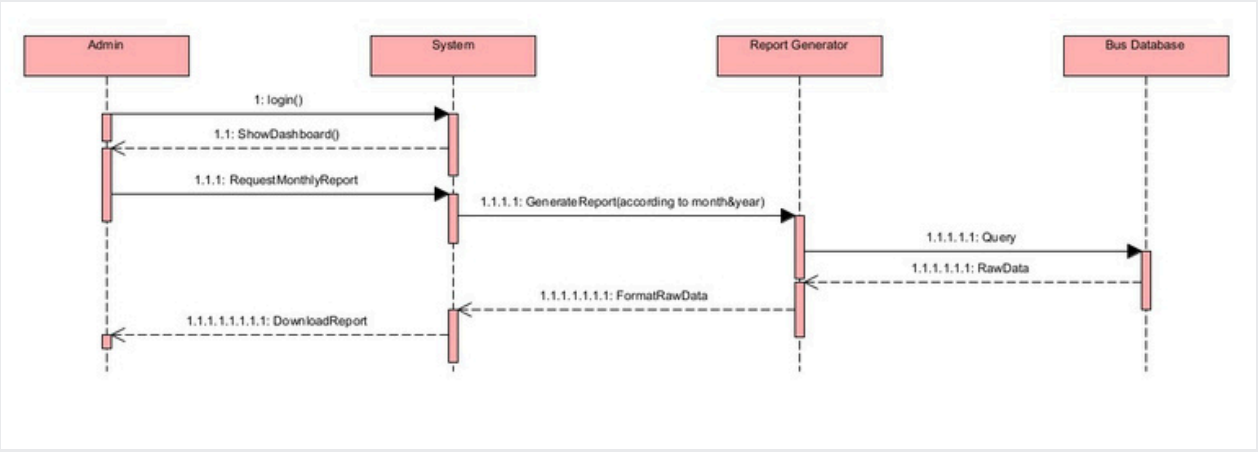
4.1.3 Student Update Details



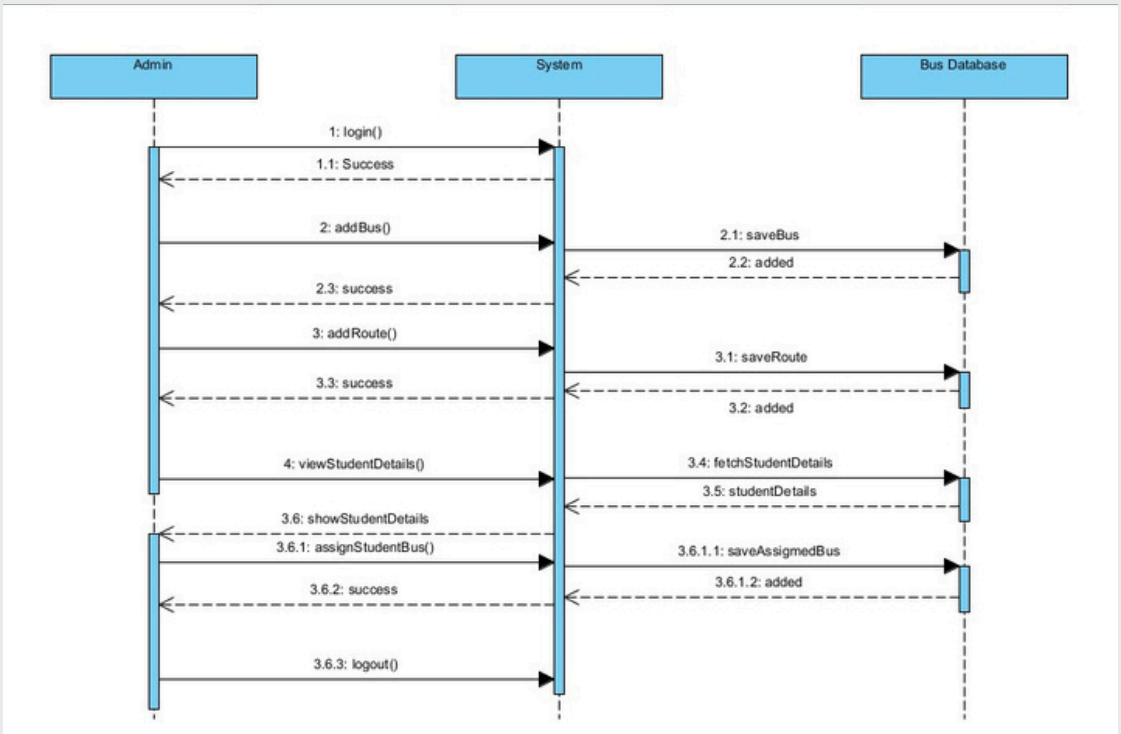
4.1.4 Add/Update Bus status



4.1.5 Admin Report Generation

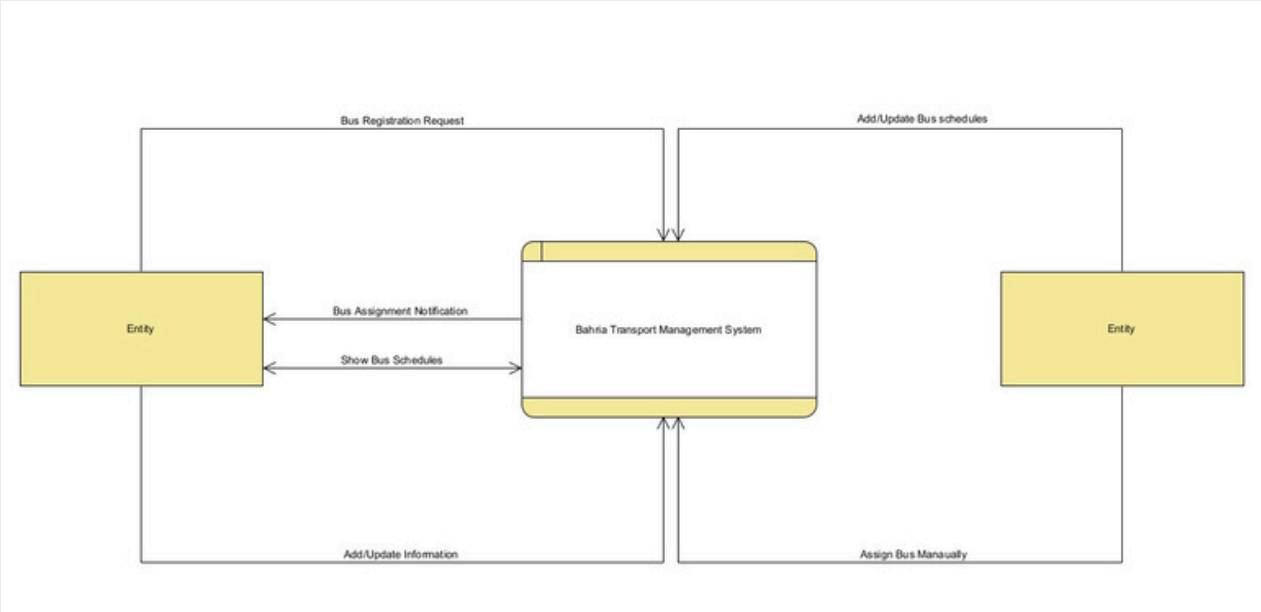


4.1.6 Admin Management

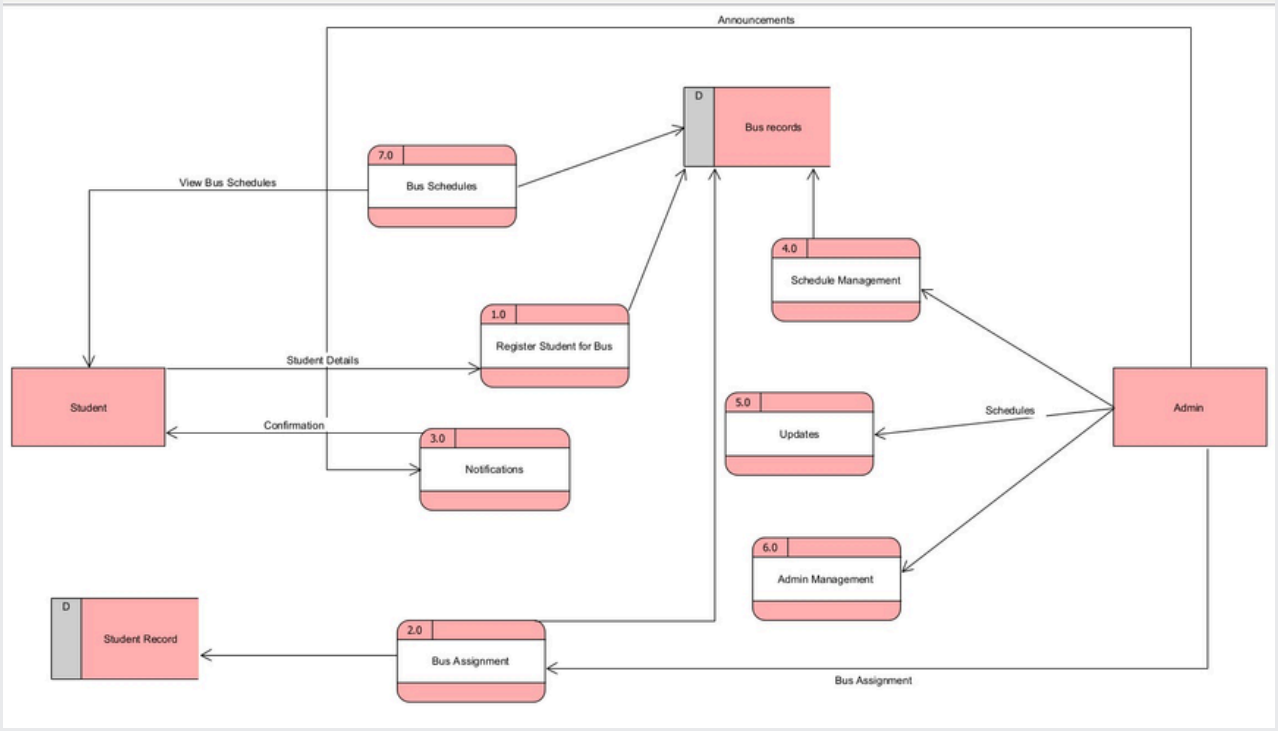


4.2 Data Flow Diagrams (DFD)

Level 0 (Context level Diagram)



Level 1



5. Change Management Process

Changes to requirements must be:

- Proposed in writing
- Reviewed by all team members
- Approved by instructor
- Documented in revision history

Changes can be submitted by any team member via email or team meeting minutes. Approval requires consensus of the team and instructor.

A. Appendices

A.1 Appendix 1: Front Page

```
==================================================
      WELCOME TO  BAHRIA TRANSPORT MANAGEMENT SYSTEM
==================================================


Select your role:
1. Admin
2. Student
3. Exit
Enter your role:
```

A.2 Appendix 2: Trello